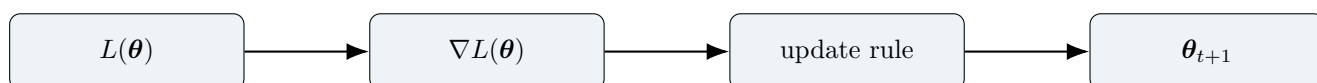


Optimization Algorithms for Neural Networks

SGD, Momentum, Adaptive Learning Rates, AdaGrad, RMSProp, Adam, Conjugate Gradient, and BFGS



A mathematical and practical guide with worked examples and optimization-path visualizations

August 2026

Contents

1	The unifying optimization problem	3
1.1	Gradient descent as the baseline	3
2	Stochastic Gradient Descent (SGD)	3
2.1	From the full gradient to a stochastic estimate	4
2.2	Why stochasticity is useful	4
2.3	Batch, iteration, and epoch	5
2.4	Numerical SGD example	6
2.5	Learning-rate schedules for SGD	6
3	Momentum	6
3.1	Motivation: oscillation in narrow valleys	6
3.2	Physical analogy	7
3.3	Worked momentum example	7
3.4	Nesterov momentum	8
4	Adaptive learning rates: the general idea	8
5	AdaGrad	9
5.1	Update equations	9
5.2	Why AdaGrad is attractive for sparse features	9
5.3	Worked AdaGrad example	10
5.4	Main weakness	11
6	RMSProp	11
6.1	Exponential memory of squared gradients	11
6.2	Interpretation	11
6.3	Worked RMSProp example	11
7	Adam	12
7.1	Combining momentum and adaptive scaling	12
7.2	Why bias correction is necessary	12
7.3	A complete first Adam step	12
7.4	Adam versus RMSProp	13
7.5	AdamW and weight decay	13
8	A unified view of SGD, Momentum, AdaGrad, RMSProp, and Adam	13
9	Curvature: why first-order methods can struggle	14
10	Conjugate Gradient (CG)	15
10.1	The linear-system perspective	15
10.2	Why ordinary gradient descent can repeat work	16
10.3	Classical CG recursion	16
10.4	Two-dimensional worked CG example	16
10.5	Nonlinear conjugate gradient	17
11	BFGS	17

11.1 Quasi-Newton idea	17
11.2 Inverse-Hessian BFGS update	18
11.3 Why BFGS can become much faster near a smooth minimum	18
11.4 Curvature condition	18
11.5 Memory cost and L-BFGS	18
12 CG and BFGS on a curved objective	18
13 Line search versus fixed learning rates	19
14 First-order adaptive optimizers versus quasi-Newton methods	20
15 Why Adam is common in deep learning but BFGS is not	20
15.1 Mini-batch noise	20
15.2 Dimension	20
15.3 Cost per iteration	20
15.4 Nonstationary stochastic objective seen by mini-batches	21
16 Condition number and optimization difficulty	21
17 The role of normalization and parameter scale	21
18 Regularization and optimizer updates	21
19 Gradient clipping	22
20 How to diagnose an optimizer during training	22
21 Hyperparameters and what they control	23
21.1 SGD	23
21.2 Momentum	23
21.3 RMSProp and Adam	23
21.4 BFGS and CG	23
22 Worked side-by-side scalar example	23
22.1 SGD	23
22.2 Momentum	24
22.3 RMSProp	24
22.4 Adam	24
23 Implementation patterns in PyTorch	24
24 When to choose which optimizer	25
25 A conceptual hierarchy	25
26 Mathematical summary sheet	26
27 Final intuition	27

1. The unifying optimization problem

Training a neural network means choosing parameters θ —all weights and biases—to minimize an empirical loss. For a dataset of N observations,

$$J(\theta) = \frac{1}{N} \sum_{i=1}^N \ell_i(\theta), \quad (1)$$

where ℓ_i is the loss contributed by observation i . The learning problem is

$$\boxed{\theta^* = \arg \min_{\theta} J(\theta).} \quad (2)$$

Backpropagation computes derivatives of the loss with respect to parameters. An optimizer answers a different question: *once the gradient is available, how should it be converted into a parameter update?*

Most methods in this note can be written schematically as

$$\boxed{\theta_{t+1} = \theta_t + \Delta \theta_t.} \quad (3)$$

The algorithms differ primarily in how they construct $\Delta \theta_t$.

Three levels that should not be confused

Backpropagation computes derivatives.

The optimizer transforms those derivatives into an update.

The learning rate or line search controls how far the algorithm moves along the chosen direction.

1.1 Gradient descent as the baseline

The first-order Taylor expansion around the current point is

$$J(\theta + \Delta \theta) \approx J(\theta) + \nabla J(\theta)^T \Delta \theta. \quad (4)$$

The direction that locally reduces the loss most rapidly per unit Euclidean distance is the negative gradient,

$$p_t = -\nabla J(\theta_t). \quad (5)$$

Classical gradient descent uses

$$\boxed{\theta_{t+1} = \theta_t - \eta_t \nabla J(\theta_t),} \quad (6)$$

with learning rate $\eta_t > 0$.

This formula is deceptively simple. The rest of the note can be read as a sequence of increasingly sophisticated answers to three problems:

- the full gradient may be expensive to compute,
- the same learning rate can be inappropriate in different directions,
- the negative gradient can be a poor search direction in narrow or curved valleys.

2. Stochastic Gradient Descent (SGD)

2.1 From the full gradient to a stochastic estimate

The exact gradient of the empirical loss is

$$\nabla J(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^N \nabla \ell_i(\boldsymbol{\theta}). \quad (7)$$

Computing it requires a pass through all N samples before every update. For modern datasets this is wasteful.

SGD replaces the full gradient with an estimate from a randomly selected mini-batch B_t of size m :

$$\mathbf{g}_t = \frac{1}{m} \sum_{i \in B_t} \nabla \ell_i(\boldsymbol{\theta}_t). \quad (8)$$

The update becomes

$$\boxed{\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \mathbf{g}_t.} \quad (9)$$

If the mini-batch is sampled uniformly, then under standard sampling assumptions

$$\mathbb{E}[\mathbf{g}_t \mid \boldsymbol{\theta}_t] = \nabla J(\boldsymbol{\theta}_t), \quad (10)$$

so the mini-batch gradient is an unbiased estimate of the full gradient.

2.2 Why stochasticity is useful

The gradient can be decomposed as

$$\mathbf{g}_t = \nabla J(\boldsymbol{\theta}_t) + \boldsymbol{\epsilon}_t, \quad \mathbb{E}[\boldsymbol{\epsilon}_t] = 0. \quad (11)$$

The update is therefore

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \nabla J(\boldsymbol{\theta}_t) - \eta_t \boldsymbol{\epsilon}_t. \quad (12)$$

The final term is gradient noise. It causes the jagged path typically associated with SGD.

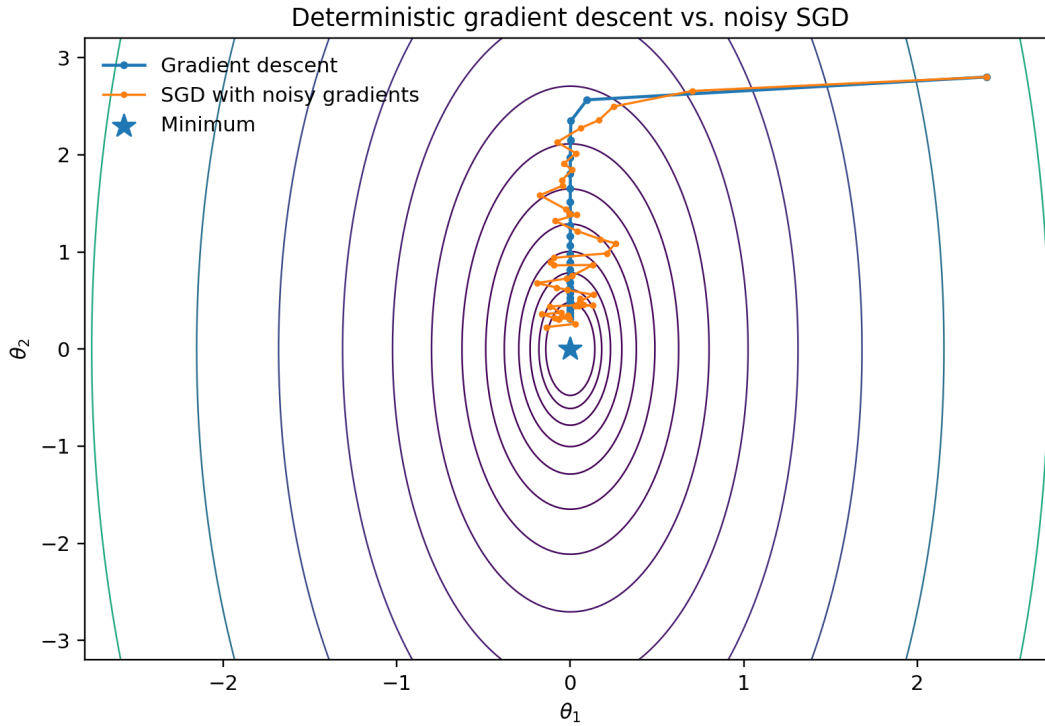


Figure 1: On a quadratic objective, full gradient descent follows a deterministic path. SGD uses noisy gradient estimates, so its path fluctuates around the deterministic direction while generally progressing toward the same minimum.

For independent observations, the variance of a mini-batch mean approximately scales as

$$\text{Var}(\mathbf{g}_t) \propto \frac{1}{m}. \quad (13)$$

Thus increasing batch size reduces gradient noise, but it also reduces the number of parameter updates obtained per pass through the data.

2.3 Batch, iteration, and epoch

If there are $N = 60,000$ observations and batch size $m = 100$, then one epoch contains approximately

$$\frac{60,000}{100} = 600 \quad (14)$$

mini-batches and therefore 600 parameter updates. Twenty epochs correspond to roughly 12,000 updates.

Terminology

Iteration / optimizer step: one parameter update.

Mini-batch: the observations used to form one gradient estimate.

Epoch: one complete traversal of the training dataset.

2.4 Numerical SGD example

Suppose the current parameter vector and mini-batch gradient are

$$\boldsymbol{\theta}_t = \begin{bmatrix} 1.20 \\ -0.50 \end{bmatrix}, \quad \mathbf{g}_t = \begin{bmatrix} 0.80 \\ -0.30 \end{bmatrix}, \quad (15)$$

with $\eta = 0.1$. Then

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - 0.1\mathbf{g}_t \quad (16)$$

$$= \begin{bmatrix} 1.20 \\ -0.50 \end{bmatrix} - 0.1 \begin{bmatrix} 0.80 \\ -0.30 \end{bmatrix} \quad (17)$$

$$= \boxed{\begin{bmatrix} 1.12 \\ -0.47 \end{bmatrix}}. \quad (18)$$

If the next mini-batch produces a different gradient, the update direction changes immediately.

2.5 Learning-rate schedules for SGD

A fixed learning rate can work, but decreasing schedules are common because early training benefits from large steps whereas late training benefits from smaller fluctuations around a minimum. Examples include

$$\text{step decay: } \eta_t = \eta_0 \gamma^{\lfloor t/T \rfloor}, \quad (19)$$

$$\text{inverse time: } \eta_t = \frac{\eta_0}{1 + kt}, \quad (20)$$

$$\text{exponential: } \eta_t = \eta_0 e^{-kt}, \quad (21)$$

$$\text{cosine: } \eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos \frac{\pi t}{T} \right). \quad (22)$$

These are *global* schedules: every parameter receives the same scalar learning rate at a given time. AdaGrad, RMSProp, and Adam go further by constructing coordinate-specific effective learning rates.

3. Momentum

3.1 Motivation: oscillation in narrow valleys

Consider a quadratic objective

$$J(\boldsymbol{\theta}) = \frac{1}{2} \boldsymbol{\theta}^T A \boldsymbol{\theta}, \quad (23)$$

where A has eigenvalues of very different magnitudes. Along a high-curvature direction, the gradient changes rapidly and plain gradient descent may oscillate. Along a low-curvature direction, progress can be slow.

Momentum accumulates a moving average of past descent directions. One common convention is

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \mathbf{g}_t, \quad (24)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \mathbf{v}_t, \quad (25)$$

where $0 \leq \beta < 1$. Another convention absorbs $(1 - \beta)$ into the moving average; both encode the same idea but differ in scaling.

Expanding the recursion,

$$\mathbf{v}_t = \mathbf{g}_t + \beta \mathbf{g}_{t-1} + \beta^2 \mathbf{g}_{t-2} + \cdots. \quad (26)$$

Thus recent gradients receive the greatest weight, and older gradients decay geometrically.

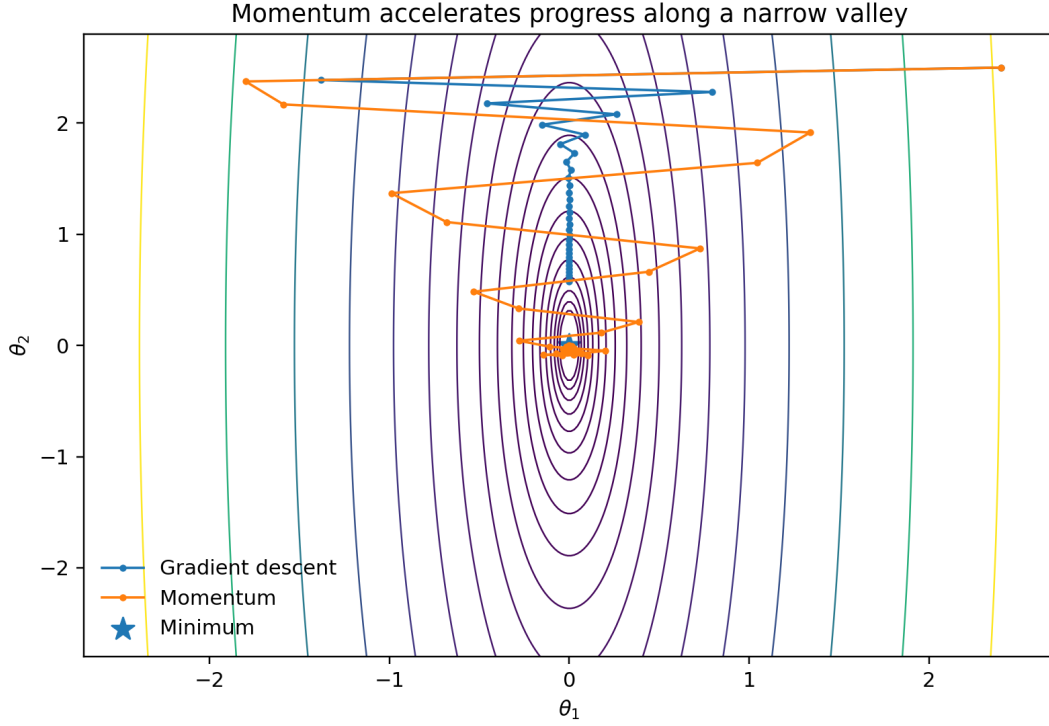


Figure 2: Momentum uses gradient history to accelerate along the shallow direction. It may overshoot across the steep direction, but with damping those oscillations decay while persistent motion accumulates.

3.2 Physical analogy

Plain SGD behaves like repeatedly placing an object in the direction of the current force with no memory. Momentum introduces a velocity. If gradients keep pointing in a similar direction, velocity accumulates. If gradients alternate sign across a valley, their contributions partly cancel.

3.3 Worked momentum example

Let

$$\eta = 0.1, \quad \beta = 0.9, \quad \mathbf{v}_0 = \mathbf{0}. \quad (27)$$

Suppose the first gradient is

$$\mathbf{g}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix}. \quad (28)$$

Then

$$\mathbf{v}_1 = 0.9\mathbf{0} + \mathbf{g}_1 = \begin{bmatrix} 2 \\ 1 \end{bmatrix} \quad (29)$$

and

$$\Delta\boldsymbol{\theta}_1 = -0.1\mathbf{v}_1 = \begin{bmatrix} -0.2 \\ -0.1 \end{bmatrix}. \quad (30)$$

Suppose the next gradient is

$$\mathbf{g}_2 = \begin{bmatrix} 1.5 \\ -0.8 \end{bmatrix}. \quad (31)$$

Then

$$\mathbf{v}_2 = 0.9 \begin{bmatrix} 2 \\ 1 \end{bmatrix} + \begin{bmatrix} 1.5 \\ -0.8 \end{bmatrix} \quad (32)$$

$$= \begin{bmatrix} 3.3 \\ 0.1 \end{bmatrix}. \quad (33)$$

The second coordinate almost cancels because its gradient reversed sign, whereas the first coordinate continues to accelerate because its gradient remained positive.

3.4 Nesterov momentum

Nesterov accelerated gradient evaluates the gradient at a look-ahead position. In one common notation,

$$\tilde{\boldsymbol{\theta}}_t = \boldsymbol{\theta}_t - \eta\beta\mathbf{v}_{t-1}, \quad (34)$$

$$\mathbf{v}_t = \beta\mathbf{v}_{t-1} + \nabla J(\tilde{\boldsymbol{\theta}}_t), \quad (35)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta\mathbf{v}_t. \quad (36)$$

The intuition is that the optimizer first asks where existing momentum would take it, then corrects the direction using the gradient at that look-ahead location.

4. Adaptive learning rates: the general idea

A single scalar learning rate assumes the same scale is appropriate for every parameter. In high-dimensional models this is often unrealistic. If coordinate j has historically large gradients, a large global step can be unstable; if another coordinate receives tiny or rare gradients, the same global learning rate can make it move too slowly.

A general adaptive update can be written

$$\boxed{\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta D_t^{-1} \mathbf{g}_t}, \quad (37)$$

where D_t is typically a positive diagonal matrix based on the history of squared gradients. Each parameter gets an effective learning rate

$$\eta_{t,j}^{\text{eff}} = \frac{\eta}{D_{t,jj}}. \quad (38)$$

This can be interpreted as a cheap diagonal preconditioner: the coordinate system is rescaled so that steep and flat directions are treated differently.

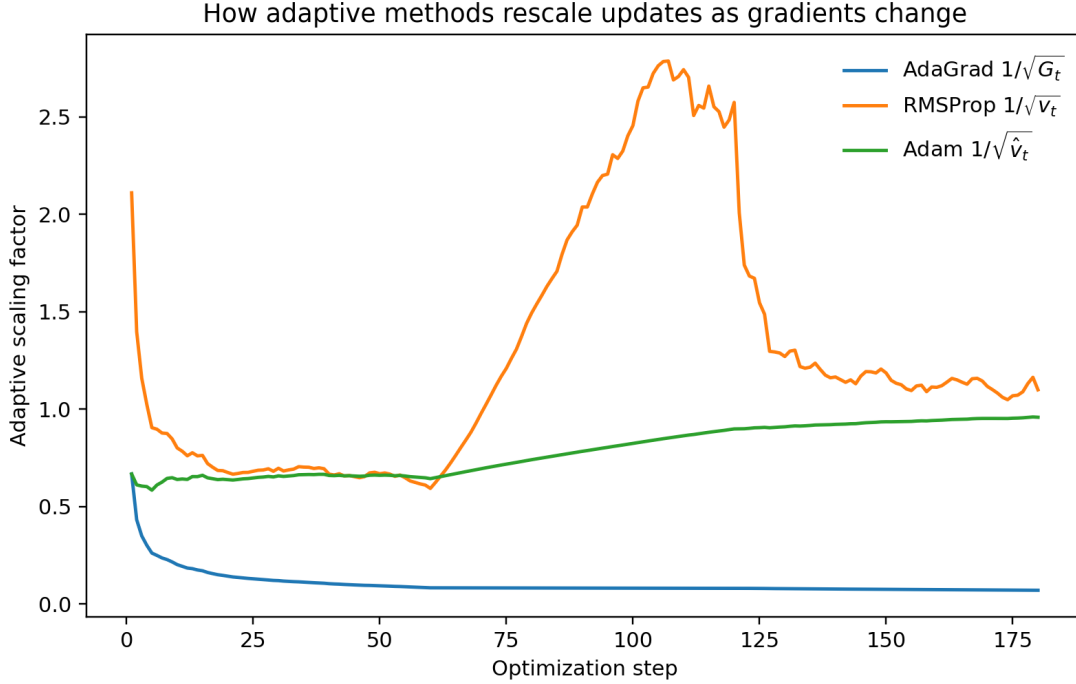


Figure 3: AdaGrad, RMSProp, and Adam all use gradient magnitude history to rescale updates, but they remember history differently. AdaGrad accumulates forever; RMSProp and Adam use exponentially decaying memory.

5. AdaGrad

5.1 Update equations

AdaGrad accumulates every past squared gradient coordinate-wise:

$$\mathbf{G}_t = \mathbf{G}_{t-1} + \mathbf{g}_t \odot \mathbf{g}_t, \quad (39)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \frac{\mathbf{g}_t}{\sqrt{\mathbf{G}_t + \epsilon}}, \quad (40)$$

where all division and square-root operations are elementwise and $\epsilon > 0$ prevents division by zero.

For coordinate j ,

$$\theta_{t+1,j} = \theta_{t,j} - \frac{\eta}{\sqrt{\sum_{s=1}^t g_{s,j}^2 + \epsilon}} g_{t,j}. \quad (41)$$

Thus the effective learning rate is

$$\boxed{\eta_{t,j}^{\text{eff}} = \frac{\eta}{\sqrt{\sum_{s=1}^t g_{s,j}^2 + \epsilon}}}. \quad (42)$$

Because the denominator can only increase, the effective learning rate is non-increasing.

5.2 Why AdaGrad is attractive for sparse features

Suppose parameter θ_1 receives a nonzero gradient almost every step while θ_2 receives one only occasionally. Then $G_{t,1}$ grows rapidly and $G_{t,2}$ grows slowly. AdaGrad automatically gives the sparse coordinate a larger

effective step.

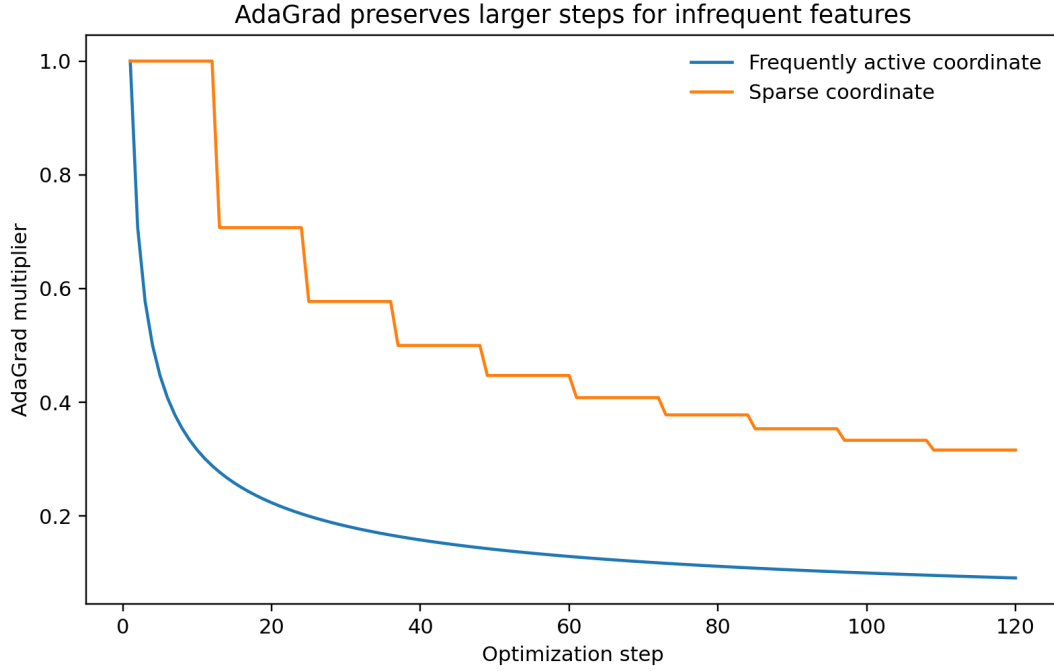


Figure 4: A coordinate receiving frequent gradients accumulates squared-gradient history rapidly, causing its AdaGrad multiplier to shrink. A sparse coordinate retains a larger multiplier for much longer.

5.3 Worked AdaGrad example

Let

$$\eta = 1, \quad \epsilon \approx 0, \quad \mathbf{G}_0 = \mathbf{0}. \quad (43)$$

Suppose

$$\mathbf{g}_1 = \begin{bmatrix} 3 \\ 1 \end{bmatrix}. \quad (44)$$

Then

$$\mathbf{G}_1 = \begin{bmatrix} 9 \\ 1 \end{bmatrix}, \quad (45)$$

and the scaled gradient is

$$\frac{\mathbf{g}_1}{\sqrt{\mathbf{G}_1}} = \begin{bmatrix} 3/3 \\ 1/1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}. \quad (46)$$

Now suppose

$$\mathbf{g}_2 = \begin{bmatrix} 3 \\ 0 \end{bmatrix}. \quad (47)$$

Then

$$\mathbf{G}_2 = \begin{bmatrix} 18 \\ 1 \end{bmatrix}, \quad (48)$$

so the first coordinate update is scaled by

$$\frac{3}{\sqrt{18}} \approx 0.707, \quad (49)$$

while the second coordinate does not move at all on this step. Repeated activity in coordinate 1 automatically shrinks its future step sizes.

5.4 Main weakness

AdaGrad's accumulator never forgets. After sufficiently many iterations, G_t can become very large and the effective learning rate can become so small that learning practically stops. RMSProp was motivated in part by replacing the cumulative sum with a moving average.

6. RMSProp

6.1 Exponential memory of squared gradients

RMSProp maintains an exponentially weighted moving average of squared gradients:

$$\mathbf{v}_t = \rho \mathbf{v}_{t-1} + (1 - \rho)(\mathbf{g}_t \odot \mathbf{g}_t), \quad (50)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \frac{\mathbf{g}_t}{\sqrt{\mathbf{v}_t} + \epsilon}. \quad (51)$$

Typical ρ values are close to 1, such as 0.9 or 0.99.

Expanding the recursion,

$$\mathbf{v}_t = (1 - \rho) \sum_{k=0}^{t-1} \rho^k (\mathbf{g}_{t-k} \odot \mathbf{g}_{t-k}), \quad (52)$$

ignoring initialization terms. Unlike AdaGrad, old squared gradients receive exponentially smaller weight.

6.2 Interpretation

If recent gradients in coordinate j are large, $v_{t,j}$ is large and the optimizer reduces the effective step in that coordinate. If recent gradients become smaller, $v_{t,j}$ can decrease again because old history decays. This makes RMSProp responsive to nonstationary gradient scales.

6.3 Worked RMSProp example

Take

$$\rho = 0.9, \quad \eta = 0.1, \quad v_0 = 0 \quad (53)$$

for one scalar parameter. If $g_1 = 2$, then

$$v_1 = 0.9(0) + 0.1(4) = 0.4. \quad (54)$$

The update magnitude is approximately

$$0.1 \frac{2}{\sqrt{0.4}} \approx 0.316. \quad (55)$$

If the next gradient is $g_2 = 1$,

$$v_2 = 0.9(0.4) + 0.1(1) = 0.46, \quad (56)$$

and the update magnitude is

$$0.1 \frac{1}{\sqrt{0.46}} \approx 0.147. \quad (57)$$

The denominator reflects recent gradient scale rather than an ever-growing sum.

7. Adam

7.1 Combining momentum and adaptive scaling

Adam—Adaptive Moment Estimation—combines two exponential moving averages:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad (58)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) (\mathbf{g}_t \odot \mathbf{g}_t). \quad (59)$$

The first moment $\mathbf{m}_t$ behaves like momentum; the second moment $\mathbf{v}_t$ tracks gradient magnitude.

Because both are initialized at zero, they are biased toward zero early in training. Adam applies bias corrections:

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad (60)$$

$$\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}. \quad (61)$$

The parameter update is

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}. \quad (62)$$

Common defaults are

$$\beta_1 = 0.9, \quad \beta_2 = 0.999, \quad \epsilon = 10^{-8}. \quad (63)$$

7.2 Why bias correction is necessary

For a constant scalar gradient g and $m_0 = 0$,

$$m_t = (1 - \beta_1^t)g. \quad (64)$$

At $t = 1$, if $\beta_1 = 0.9$, then $m_1 = 0.1g$, which severely underestimates the true first moment. Dividing by $1 - \beta_1^t$ yields

$$\hat{m}_t = g. \quad (65)$$

The same logic applies to v_t .

7.3 A complete first Adam step

Let a scalar gradient be $g_1 = 0.4$, with

$$\beta_1 = 0.9, \quad \beta_2 = 0.999, \quad \eta = 0.001. \quad (66)$$

Starting from $m_0 = v_0 = 0$,

$$m_1 = 0.9(0) + 0.1(0.4) = 0.04, \quad (67)$$

$$v_1 = 0.999(0) + 0.001(0.4^2) = 0.00016. \quad (68)$$

Bias correction gives

$$\hat{m}_1 = \frac{0.04}{1 - 0.9} = 0.4, \quad (69)$$

$$\hat{v}_1 = \frac{0.00016}{1 - 0.999} = 0.16. \quad (70)$$

Hence

$$\Delta\theta_1 \approx -0.001 \frac{0.4}{\sqrt{0.16}} = -0.001. \quad (71)$$

At the first step, the magnitude is therefore close to the base learning rate when ϵ is negligible and the gradient is nonzero.

7.4 Adam versus RMSProp

RMSProp uses the current gradient in the numerator and an exponential average of squared gradients in the denominator:

$$\Delta\theta_t^{\text{RMSProp}} \propto -\frac{\mathbf{g}_t}{\sqrt{\mathbf{v}_t}}. \quad (72)$$

Adam replaces the numerator with a momentum-like first moment:

$$\Delta\theta_t^{\text{Adam}} \propto -\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t}}. \quad (73)$$

Thus Adam smooths both the *direction* and the *scale* of the gradient.

7.5 AdamW and weight decay

A common modern variant is AdamW, which decouples weight decay from the adaptive gradient update. Schematically,

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} - \eta\lambda\theta_t. \quad (74)$$

This is different from simply adding $\lambda\theta$ to the gradient before Adam rescales it, because adaptive coordinate-wise scaling would then distort the regularization term.

8. A unified view of SGD, Momentum, AdaGrad, RMSProp, and Adam

All five methods use first derivatives, but they process them differently.

Method	State remembered	Core idea
SGD	None beyond current parameters	Follow the current mini-batch gradient
Momentum	Exponential history of gradients / velocity	Smooth directions and accumulate persistent motion
AdaGrad	Cumulative squared gradients	Shrink learning rates in coordinates that have received large/frequent gradients
RMSProp	Exponential history of squared gradients	Adapt to recent coordinate-wise gradient scale without permanent decay
Adam	Exponential first and second moments	Momentum-like direction plus RMS-style adaptive coordinate scaling

A useful generic representation is

$$\Delta\boldsymbol{\theta}_t = -\eta_t P_t \mathbf{d}_t, \quad (75)$$

where $\mathbf{d}_t$ is a direction estimate and P_t is a scaling or preconditioning matrix. Then:

- SGD: $\mathbf{d}_t = \mathbf{g}_t$, $P_t = I$,
- Momentum: $\mathbf{d}_t$ contains gradient history, $P_t = I$,
- AdaGrad/RMSProp: $\mathbf{d}_t = \mathbf{g}_t$, P_t is diagonal and adaptive,
- Adam: both $\mathbf{d}_t$ and P_t depend on gradient history.

9. Curvature: why first-order methods can struggle

The gradient gives slope but not curvature. A second-order Taylor expansion is

$$J(\boldsymbol{\theta} + \mathbf{p}) \approx J(\boldsymbol{\theta}) + \nabla J(\boldsymbol{\theta})^T \mathbf{p} + \frac{1}{2} \mathbf{p}^T H \mathbf{p}, \quad (76)$$

where

$$H = \nabla^2 J(\boldsymbol{\theta}) \quad (77)$$

is the Hessian matrix.

Minimizing this local quadratic model with respect to $\mathbf{p}$ yields the Newton direction:

$$\nabla J + H \mathbf{p} = 0, \quad (78)$$

$$\boxed{\mathbf{p}_N = -H^{-1} \nabla J.} \quad (79)$$

The Hessian rotates and rescales the gradient according to local curvature.

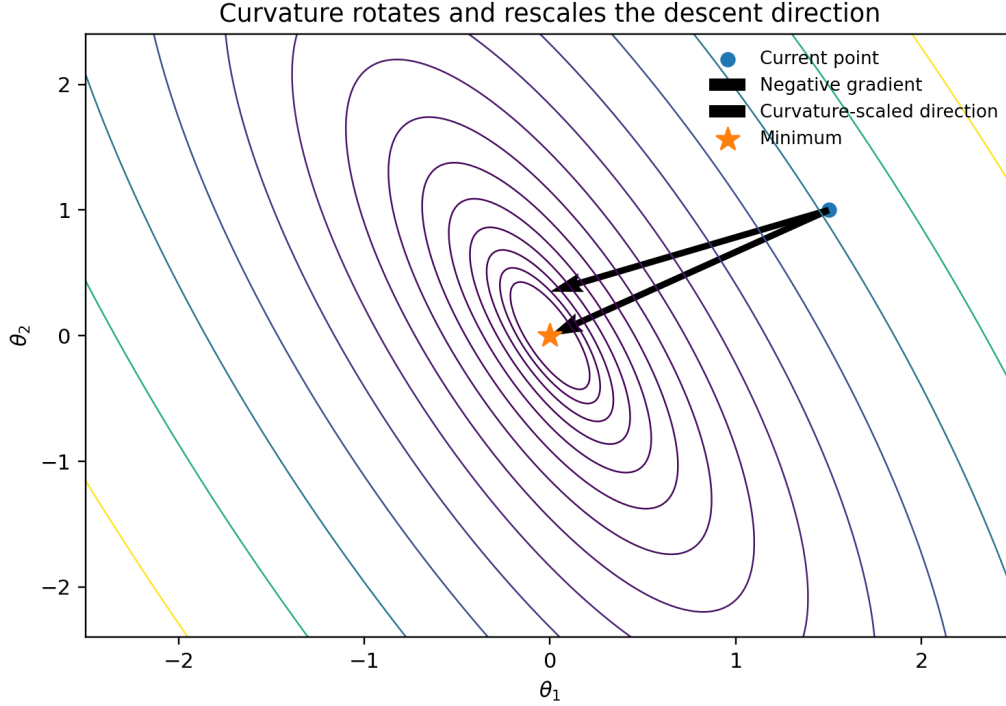


Figure 5: The negative gradient is steepest in Euclidean distance, but a curvature-scaled direction can point much more directly toward the minimum of an anisotropic quadratic.

The problem is cost. For d parameters, a dense Hessian contains d^2 entries, and explicitly forming or inverting it is infeasible for large neural networks. Conjugate Gradient and BFGS address curvature in different ways.

10. Conjugate Gradient (CG)

10.1 The linear-system perspective

Classical Conjugate Gradient solves a symmetric positive-definite linear system

$$A\mathbf{x} = \mathbf{b}. \quad (80)$$

Equivalently, it minimizes the quadratic

$$f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T A\mathbf{x} - \mathbf{b}^T \mathbf{x}. \quad (81)$$

Its gradient is

$$\nabla f(\mathbf{x}) = A\mathbf{x} - \mathbf{b}. \quad (82)$$

The residual

$$\mathbf{r}_k = \mathbf{b} - A\mathbf{x}_k = -\nabla f(\mathbf{x}_k) \quad (83)$$

is therefore the negative gradient.

10.2 Why ordinary gradient descent can repeat work

Steepest descent chooses each new direction from the local gradient, but successive directions can undo components of previous progress in elongated quadratics. CG constructs directions that are A -conjugate:

$$\boxed{\mathbf{p}_i^T A \mathbf{p}_j = 0 \quad (i \neq j).} \quad (84)$$

For a quadratic, moving optimally along one conjugate direction does not spoil the optimum already achieved along previous conjugate directions.

10.3 Classical CG recursion

Starting from $\mathbf{x}_0$, define

$$\mathbf{r}_0 = \mathbf{b} - A\mathbf{x}_0, \quad (85)$$

$$\mathbf{p}_0 = \mathbf{r}_0. \quad (86)$$

Then repeat

$$\alpha_k = \frac{\mathbf{r}_k^T \mathbf{r}_k}{\mathbf{p}_k^T A \mathbf{p}_k}, \quad (87)$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k, \quad (88)$$

$$\mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k A \mathbf{p}_k, \quad (89)$$

$$\beta_k = \frac{\mathbf{r}_{k+1}^T \mathbf{r}_{k+1}}{\mathbf{r}_k^T \mathbf{r}_k}, \quad (90)$$

$$\mathbf{p}_{k+1} = \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k. \quad (91)$$

In exact arithmetic, a d -dimensional positive-definite quadratic is solved in at most d CG iterations.

10.4 Two-dimensional worked CG example

Let

$$A = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad \mathbf{x}_0 = \begin{bmatrix} 0 \\ 0 \end{bmatrix}. \quad (92)$$

Then

$$\mathbf{r}_0 = \mathbf{b} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad \mathbf{p}_0 = \mathbf{r}_0. \quad (93)$$

Compute

$$A\mathbf{p}_0 = \begin{bmatrix} 6 \\ 7 \end{bmatrix}. \quad (94)$$

Therefore

$$\alpha_0 = \frac{1^2 + 2^2}{[1, 2][6, 7]^T} = \frac{5}{20} = 0.25. \quad (95)$$

Hence

$$\mathbf{x}_1 = \begin{bmatrix} 0.25 \\ 0.5 \end{bmatrix}, \quad (96)$$

and

$$\mathbf{r}_1 = \begin{bmatrix} 1 \\ 2 \end{bmatrix} - 0.25 \begin{bmatrix} 6 \\ 7 \end{bmatrix} = \begin{bmatrix} -0.5 \\ 0.25 \end{bmatrix}. \quad (97)$$

Then

$$\beta_0 = \frac{(-0.5)^2 + 0.25^2}{1^2 + 2^2} = \frac{0.3125}{5} = 0.0625, \quad (98)$$

so

$$\mathbf{p}_1 = \mathbf{r}_1 + 0.0625\mathbf{p}_0 = \begin{bmatrix} -0.4375 \\ 0.375 \end{bmatrix}. \quad (99)$$

A second line minimization reaches the exact solution in exact arithmetic, consistent with the two-dimensional guarantee.

10.5 Nonlinear conjugate gradient

Neural-network losses are not quadratic, so one can use nonlinear CG. A common structure is

$$\mathbf{p}_0 = -\mathbf{g}_0, \quad (100)$$

$$\mathbf{p}_{k+1} = -\mathbf{g}_{k+1} + \beta_k \mathbf{p}_k, \quad (101)$$

combined with a line search for α_k :

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \alpha_k \mathbf{p}_k. \quad (102)$$

Different formulas for β_k lead to variants such as Fletcher–Reeves and Polak–Ribiere. Fletcher–Reeves uses

$$\beta_k^{FR} = \frac{\mathbf{g}_{k+1}^T \mathbf{g}_{k+1}}{\mathbf{g}_k^T \mathbf{g}_k}. \quad (103)$$

Polak–Ribiere uses

$$\beta_k^{PR} = \frac{\mathbf{g}_{k+1}^T (\mathbf{g}_{k+1} - \mathbf{g}_k)}{\mathbf{g}_k^T \mathbf{g}_k}. \quad (104)$$

For nonlinear problems, conjugacy is only approximate and periodic restarts are often useful.

11. BFGS

11.1 Quasi-Newton idea

Newton’s method uses

$$\mathbf{p}_k = -H_k^{-1} \mathbf{g}_k. \quad (105)$$

BFGS avoids explicitly calculating the true Hessian. Instead it iteratively constructs an approximation from changes in parameters and gradients.

Define

$$\mathbf{s}_k = \boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_k, \quad (106)$$

$$\mathbf{y}_k = \mathbf{g}_{k+1} - \mathbf{g}_k. \quad (107)$$

For a locally quadratic function, the Hessian approximately satisfies

$$\mathbf{y}_k \approx H \mathbf{s}_k. \quad (108)$$

BFGS updates an approximation so that it satisfies the secant condition

$$B_{k+1}\mathbf{s}_k = \mathbf{y}_k. \quad (109)$$

11.2 Inverse-Hessian BFGS update

It is common to maintain H_k , an approximation to the *inverse* Hessian. Let

$$\rho_k = \frac{1}{\mathbf{y}_k^T \mathbf{s}_k}. \quad (110)$$

Then

$$H_{k+1} = (I - \rho_k \mathbf{s}_k \mathbf{y}_k^T) H_k (I - \rho_k \mathbf{y}_k \mathbf{s}_k^T) + \rho_k \mathbf{s}_k \mathbf{s}_k^T. \quad (111)$$

The search direction is

$$\mathbf{p}_k = -H_k \mathbf{g}_k. \quad (112)$$

A line search then selects α_k , and

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k + \alpha_k \mathbf{p}_k. \quad (113)$$

11.3 Why BFGS can become much faster near a smooth minimum

If the inverse-Hessian approximation improves, the optimizer increasingly rescales the gradient in a way similar to Newton's method. On smooth deterministic problems BFGS can exhibit superlinear local convergence, meaning that the rate of error reduction accelerates as it approaches the solution.

11.4 Curvature condition

To preserve positive definiteness of the BFGS inverse-Hessian approximation, one typically requires

$$\mathbf{y}_k^T \mathbf{s}_k > 0. \quad (114)$$

Line searches satisfying appropriate Wolfe conditions help enforce this relationship.

11.5 Memory cost and L-BFGS

A dense BFGS matrix costs $O(d^2)$ memory. For neural networks with millions of parameters this is impossible. Limited-memory BFGS (L-BFGS) stores only the most recent m pairs

$$(\mathbf{s}_k, \mathbf{y}_k) \quad (115)$$

and implicitly computes the inverse-Hessian action. Its storage is $O(md)$, making it much more practical for larger problems.

12. CG and BFGS on a curved objective

The Rosenbrock function

$$f(x, y) = 100(y - x^2)^2 + (1 - x)^2 \quad (116)$$

is a classic difficult test because the optimum $(1, 1)$ lies in a narrow curved valley.

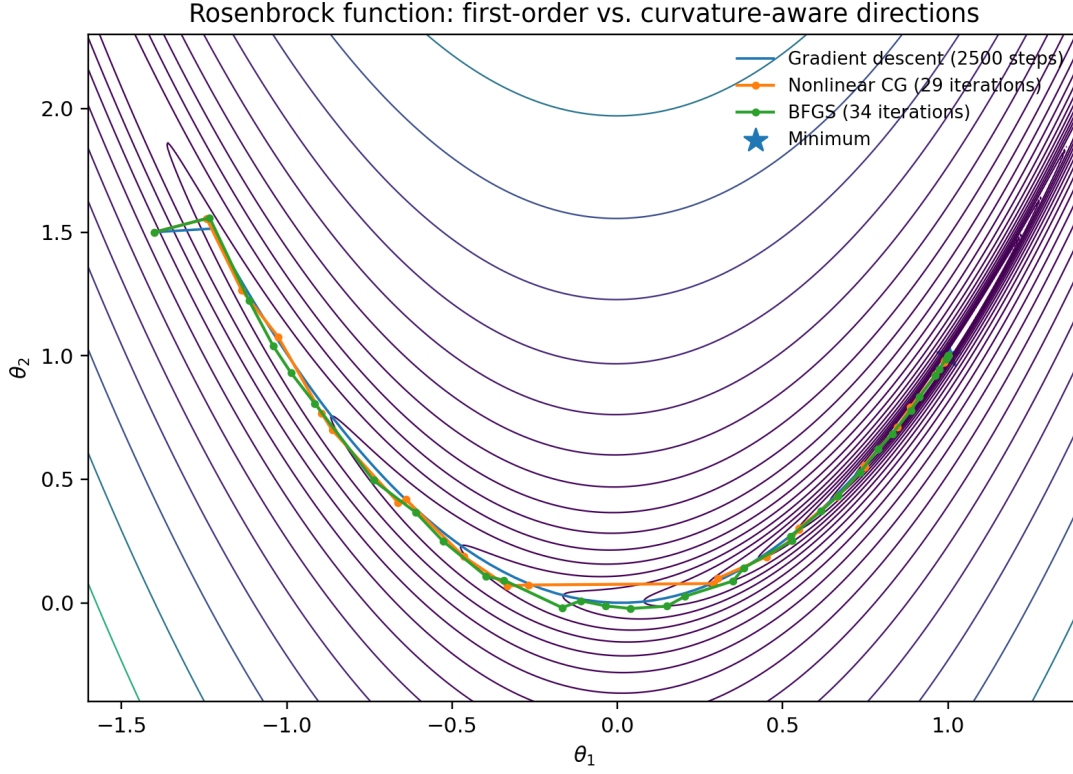


Figure 6: Illustrative trajectories on the Rosenbrock function. Gradient descent needs many small steps in the curved valley, whereas nonlinear CG and BFGS construct directions that exploit information beyond the current negative gradient. Iteration counts depend strongly on line-search and stopping settings.

This picture should not be interpreted as a universal speed ranking. It demonstrates a structural advantage on smooth deterministic objectives: curvature-aware or conjugate directions can dramatically reduce zig-zagging. Neural-network training introduces stochastic gradients, nonconvexity, huge dimension, and batch noise, which change the trade-off.

13. Line search versus fixed learning rates

SGD-family methods usually choose a learning rate directly. Classical CG and BFGS are commonly paired with a line search. Given a direction $\mathbf{p}_k$, define the one-dimensional function

$$\phi(\alpha) = J(\boldsymbol{\theta}_k + \alpha \mathbf{p}_k). \quad (117)$$

A line search tries to choose $\alpha > 0$ that gives sufficient decrease while maintaining useful curvature information.

The Armijo sufficient-decrease condition is

$$J(\boldsymbol{\theta}_k + \alpha \mathbf{p}_k) \leq J(\boldsymbol{\theta}_k) + c_1 \alpha \mathbf{g}_k^T \mathbf{p}_k, \quad (118)$$

with small $c_1 \in (0, 1)$. Wolfe-type conditions add a requirement on the directional derivative after the step.

Line searches usually need multiple function or gradient evaluations per iteration. That overhead is acceptable when each iteration is highly informative and the objective is deterministic, but becomes

expensive when every gradient evaluation means another large neural-network mini-batch or full-batch pass.

14. First-order adaptive optimizers versus quasi-Newton methods

Method	Derivative information	Memory	Typical step control	Key strength
SGD	Gradient estimate	$O(d)$	Learning rate / schedule	Cheap, scalable, strong baseline
Momentum	Gradient estimate + velocity	$O(d)$	Learning rate	Reduces oscillation; accelerates persistent directions
AdaGrad	Gradient + cumulative squares	$O(d)$	Base LR + coordinate scaling	Sparse/infrequent feature adaptation
RMSProp	Gradient + EMA of squares	$O(d)$	Base LR + coordinate scaling	Handles changing gradient scales
Adam	EMA of gradient and squares	$O(d)$	Base LR + coordinate scaling	Robust default, smooth adaptive updates
Nonlinear CG	Gradient + previous direction	$O(d)$	Usually line search	Conjugate directions without storing Hessian
BFGS	Gradients + dense inverse-Hessian approx.	$O(d^2)$	Usually line search	Strong local behavior on smooth deterministic problems
L-BFGS	Gradients + recent (s, y) pairs	$O(md)$	Usually line search	Quasi-Newton behavior with manageable memory

15. Why Adam is common in deep learning but BFGS is not

The difference is not that BFGS is mathematically inferior. In fact, on smooth deterministic moderate-dimensional problems BFGS is often extraordinarily effective. Deep learning changes the computational environment.

15.1 Mini-batch noise

BFGS infers curvature from gradient differences

$$\mathbf{y}_k = \mathbf{g}_{k+1} - \mathbf{g}_k. \quad (119)$$

If $\mathbf{g}_k$ and $\mathbf{g}_{k+1}$ are calculated from different noisy mini-batches, $\mathbf{y}_k$ contains both true curvature information and sampling noise. This can corrupt the Hessian approximation.

15.2 Dimension

A network with 10^7 parameters would require 10^{14} entries for a dense BFGS matrix. Even L-BFGS, while vastly cheaper, generally performs more global synchronization and line-search work than mini-batch Adam or SGD.

15.3 Cost per iteration

Adam uses one gradient calculation per optimizer step. BFGS/CG line searches may evaluate the objective or gradient several times for one accepted update. In large neural networks, wall-clock progress per data pass often matters more than minimizing the number of abstract optimization iterations.

15.4 Nonstationary stochastic objective seen by mini-batches

With random mini-batches, the optimizer effectively sees a slightly different empirical surface every step. Adaptive first-order methods are designed to tolerate this noisy setting. Classical quasi-Newton assumptions fit deterministic smooth objectives more naturally.

Practical consequence

For ordinary large-scale mini-batch neural networks, SGD with momentum, Adam, or AdamW are the mainstream choices. L-BFGS can still be excellent for smaller full-batch networks, scientific machine learning, deterministic fine-tuning problems, or situations where the dataset fits comfortably into memory and accurate gradients are available.

16. Condition number and optimization difficulty

For a positive-definite quadratic with Hessian A , define the condition number

$$\kappa(A) = \frac{\lambda_{\max}(A)}{\lambda_{\min}(A)}. \quad (120)$$

When $\kappa \gg 1$, contour ellipses are highly elongated. Gradient descent must use a learning rate small enough to remain stable in the steep direction, causing slow movement along the flat direction.

Adaptive methods partly address this through diagonal scaling. Newton/BFGS attempts to model more general curvature, including rotations caused by off-diagonal Hessian terms. This distinction matters:

- AdaGrad/RMSProp/Adam mainly rescale coordinates independently,
- a full Hessian or BFGS approximation can also represent interactions between coordinates.

A diagonal adaptive optimizer therefore cannot generally undo an arbitrarily rotated ill-conditioned valley as perfectly as a good full curvature model.

17. The role of normalization and parameter scale

Adaptive optimizers are often described as being insensitive to gradient scale, but this should not be overstated. Feature normalization, initialization, normalization layers, residual connections, and architecture all affect the loss geometry and therefore the optimizer's job.

For a simple linear model

$$\hat{y} = \theta_1 x_1 + \theta_2 x_2, \quad (121)$$

if x_1 is numerically 1000 times larger than x_2 , the corresponding gradients can differ enormously. Standardization can make the problem better conditioned before the optimizer is even involved.

18. Regularization and optimizer updates

Suppose the objective includes L2 regularization

$$J_{\text{reg}}(\boldsymbol{\theta}) = J(\boldsymbol{\theta}) + \frac{\lambda}{2} \|\boldsymbol{\theta}\|^2. \quad (122)$$

Then

$$\nabla J_{reg} = \nabla J + \lambda \boldsymbol{\theta}. \quad (123)$$

With plain SGD,

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta(\mathbf{g}_t + \lambda \boldsymbol{\theta}_t) \quad (124)$$

$$= (1 - \eta\lambda)\boldsymbol{\theta}_t - \eta\mathbf{g}_t. \quad (125)$$

Thus L2 regularization and multiplicative weight decay coincide for basic SGD. With adaptive methods they generally do not coincide because the optimizer rescales gradient coordinates. This motivates decoupled weight decay in AdamW.

19. Gradient clipping

Large gradients can produce unstable updates, especially in recurrent or very deep networks. Global norm clipping replaces $\mathbf{g}$ by

$$\tilde{\mathbf{g}} = \mathbf{g} \min \left(1, \frac{c}{\|\mathbf{g}\|} \right), \quad (126)$$

where c is a chosen threshold. If $\|\mathbf{g}\| \leq c$, nothing changes. Otherwise the gradient direction is preserved but its norm is reduced to c .

Clipping is conceptually separate from the optimizer: the gradient can be clipped before being passed to SGD, momentum, Adam, and so on.

20. How to diagnose an optimizer during training

Optimizer choice should be diagnosed through training behavior rather than selected by name alone.

Observation	Possible cause	Typical response
Loss explodes immediately	Learning rate too high; numerical instability	Lower LR; inspect initialization and gradient norms
Loss decreases extremely slowly	LR too small; poor conditioning; vanishing gradients	Increase LR carefully; normalize; try momentum/adaptive method
Loss oscillates strongly	LR high or narrow valley	Lower LR; add momentum; improve conditioning
Training loss good, validation poor	Overfitting rather than optimizer failure	Regularization, early stopping, more data, architecture changes
AdaGrad stalls late	Accumulator has made effective LR tiny	Use RMSProp/Adam or a different schedule
Adam trains quickly but final generalization disappoints	Optimization/generalization trade-off	Tune weight decay/LR; consider SGD momentum fine-tuning
BFGS unstable with mini-batches	Noisy curvature differences / line search	Larger or full batch; L-BFGS; use first-order method

Useful quantities to monitor include

$$J_t, \quad \|g_t\|, \quad \|\Delta\theta_t\|, \quad \frac{\|\Delta\theta_t\|}{\|\theta_t\| + \epsilon}, \quad (127)$$

as well as training and validation metrics.

21. Hyperparameters and what they control

21.1 SGD

The principal hyperparameter is the learning rate η , plus its schedule and batch size. Batch size changes not only throughput but also gradient variance.

21.2 Momentum

β controls memory. The approximate memory timescale of an exponential moving average is on the order of

$$\frac{1}{1 - \beta}. \quad (128)$$

Thus $\beta = 0.9$ corresponds roughly to a ten-step scale, while $\beta = 0.99$ has a much longer memory.

21.3 RMSProp and Adam

β_2 or ρ controls how quickly the estimate of squared-gradient scale adapts. A value very close to one is stable but slow to react to sudden changes. ϵ primarily prevents numerical division by zero but can also affect effective scaling when v_t is very small.

21.4 BFGS and CG

The line-search rule, tolerances, restart criteria, and curvature safeguards are central. Unlike Adam, the nominal method name does not fully specify practical behavior because line-search implementation can materially change convergence.

22. Worked side-by-side scalar example

Suppose a scalar parameter has gradients

$$g_1 = 2, \quad g_2 = 1, \quad g_3 = 0.5. \quad (129)$$

Take base learning rate $\eta = 0.1$. For illustration use momentum $\beta = 0.9$, RMSProp $\rho = 0.9$, and Adam $\beta_1 = 0.9, \beta_2 = 0.999$.

22.1 SGD

The update magnitudes are simply

$$0.2, \quad 0.1, \quad 0.05. \quad (130)$$

22.2 Momentum

With $v_0 = 0$ and the convention $v_t = 0.9v_{t-1} + g_t$,

$$v_1 = 2, \tag{131}$$

$$v_2 = 2.8, \tag{132}$$

$$v_3 = 3.02. \tag{133}$$

The corresponding update magnitudes are

$$0.2, \quad 0.28, \quad 0.302. \tag{134}$$

Even though the current gradient shrinks, accumulated velocity continues to carry the parameter forward.

22.3 RMSProp

With $v_0 = 0$,

$$v_1 = 0.4, \tag{135}$$

$$v_2 = 0.46, \tag{136}$$

$$v_3 = 0.439. \tag{137}$$

The approximate update magnitudes are

$$0.316, \quad 0.147, \quad 0.0755. \tag{138}$$

The denominator adapts to recent gradient scale.

22.4 Adam

Adam smooths the numerator as well as the denominator. The exact values depend on bias correction, but the key conceptual difference is that a decreasing current gradient does not immediately erase the momentum accumulated in $\hat{m}_t$, while $\hat{v}_t$ normalizes by recent squared-gradient magnitude.

This simple example shows why “the learning rate” is not the same object across optimizers. In Adam, η is a base scale multiplied by a coordinate- and time-dependent ratio.

23. Implementation patterns in PyTorch

A standard training loop separates gradient computation from parameter update:

```
optimizer.zero_grad()
pred = model(x_batch)
loss = loss_fn(pred, y_batch)
loss.backward()
optimizer.step()
```

The line

```
loss.backward()
```

computes gradients through backpropagation. The line

```
optimizer.step()
```

uses the selected optimization rule to update parameters.

Typical optimizer constructions are conceptually

```
optimizer = torch.optim.SGD(model.parameters(), lr=0.01)
optimizer = torch.optim.SGD(model.parameters(), lr=0.01, momentum=0.9)
optimizer = torch.optim.Adagrad(model.parameters(), lr=0.01)
optimizer = torch.optim.RMSprop(model.parameters(), lr=0.001, alpha=0.99)
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
optimizer = torch.optim.AdamW(model.parameters(), lr=0.001, weight_decay=0.01)
optimizer = torch.optim.LBFGS(model.parameters(), lr=1.0)
```

L-BFGS is unusual because line-search-style operation often requires a closure that can reevaluate the model and loss. This reflects the deeper algorithmic difference between quasi-Newton methods and one-gradient-per-step mini-batch optimizers.

24. When to choose which optimizer

There is no universal winner, but the following decision logic is useful.

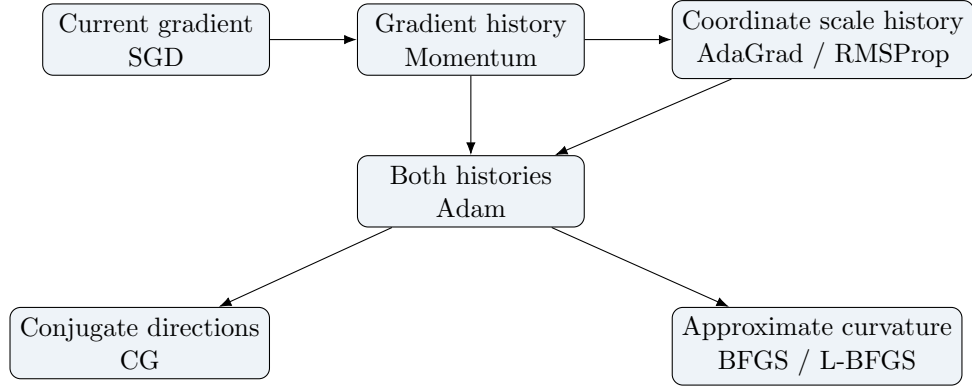
Situation	Reasonable starting point
Large standard deep-learning model	AdamW or SGD with momentum
Need a robust fast baseline with limited tuning	Adam / AdamW
Sparse features or embeddings	AdaGrad can be attractive; Adam-type methods are also common
Want strong classical generalization baseline	SGD + momentum with tuned schedule
Highly nonstationary gradient scales	RMSProp or Adam
Small smooth deterministic full-batch problem	BFGS / L-BFGS can be excellent
Quadratic linear system / Hessian-vector subproblem	Conjugate Gradient
Large problem where full Hessian is impossible but deterministic gradients are accurate	L-BFGS or nonlinear CG may be worth testing

Do not compare optimizers at one arbitrary learning rate

A poor learning rate can make a strong optimizer look bad. Fair comparison requires tuning the base learning rate and, when relevant, schedule, momentum coefficients, weight decay, and batch size. Convergence speed should be compared in wall-clock time or data passes as well as optimizer iterations.

25. A conceptual hierarchy

The methods can be organized by the information they exploit:



This is not a strict historical or superiority ranking. It is a map of what information is being retained and used.

26. Mathematical summary sheet

SGD

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta_t \mathbf{g}_t. \quad (139)$$

Momentum

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \mathbf{g}_t, \quad \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \mathbf{v}_t. \quad (140)$$

AdaGrad

$$\mathbf{G}_t = \mathbf{G}_{t-1} + \mathbf{g}_t^2, \quad \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \frac{\mathbf{g}_t}{\sqrt{\mathbf{G}_t} + \epsilon}. \quad (141)$$

RMSProp

$$\mathbf{v}_t = \rho \mathbf{v}_{t-1} + (1 - \rho) \mathbf{g}_t^2, \quad \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \frac{\mathbf{g}_t}{\sqrt{\mathbf{v}_t} + \epsilon}. \quad (142)$$

Adam

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad (143)$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2, \quad (144)$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t}, \quad (145)$$

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}. \quad (146)$$

Conjugate Gradient for $A\mathbf{x} = \mathbf{b}$

$$\alpha_k = \frac{\mathbf{r}_k^T \mathbf{r}_k}{\mathbf{p}_k^T A \mathbf{p}_k}, \quad (147)$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k, \quad (148)$$

$$\mathbf{r}_{k+1} = \mathbf{r}_k - \alpha_k A \mathbf{p}_k, \quad (149)$$

$$\beta_k = \frac{\mathbf{r}_{k+1}^T \mathbf{r}_{k+1}}{\mathbf{r}_k^T \mathbf{r}_k}, \quad (150)$$

$$\boxed{\mathbf{p}_{k+1} = \mathbf{r}_{k+1} + \beta_k \mathbf{p}_k}. \quad (151)$$

BFGS inverse-Hessian update

$$\mathbf{s}_k = \boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_k, \quad \mathbf{y}_k = \mathbf{g}_{k+1} - \mathbf{g}_k, \quad (152)$$

$$\rho_k = (\mathbf{y}_k^T \mathbf{s}_k)^{-1}, \quad (153)$$

$$\boxed{H_{k+1} = (I - \rho_k \mathbf{s}_k \mathbf{y}_k^T) H_k (I - \rho_k \mathbf{y}_k \mathbf{s}_k^T) + \rho_k \mathbf{s}_k \mathbf{s}_k^T} \quad (154)$$

27. Final intuition

The optimizers differ in what they believe about useful information from the past.

SGD says: *the current mini-batch gradient is enough; move against it.* Momentum says: *directions that have remained consistent deserve accumulated velocity.* AdaGrad says: *coordinates that have repeatedly seen large gradients should receive progressively smaller steps.* RMSProp says: *use recent, not permanent, squared-gradient history.* Adam says: *smooth both direction and scale.* Conjugate Gradient says: *choose new search directions that do not repeat work performed along previous directions.* BFGS says: *infer local curvature from how gradients change as parameters move, then use that curvature approximation to reshape the search direction.*

The common training loop remains

$$\boxed{\text{forward pass} \longrightarrow \text{loss} \longrightarrow \text{backpropagation} \longrightarrow \text{gradient} \longrightarrow \text{optimizer} \longrightarrow \text{new parameters.}} \quad (155)$$

The optimizer acts only at the final transition, but its choice can strongly determine how efficiently the network traverses the loss surface.